



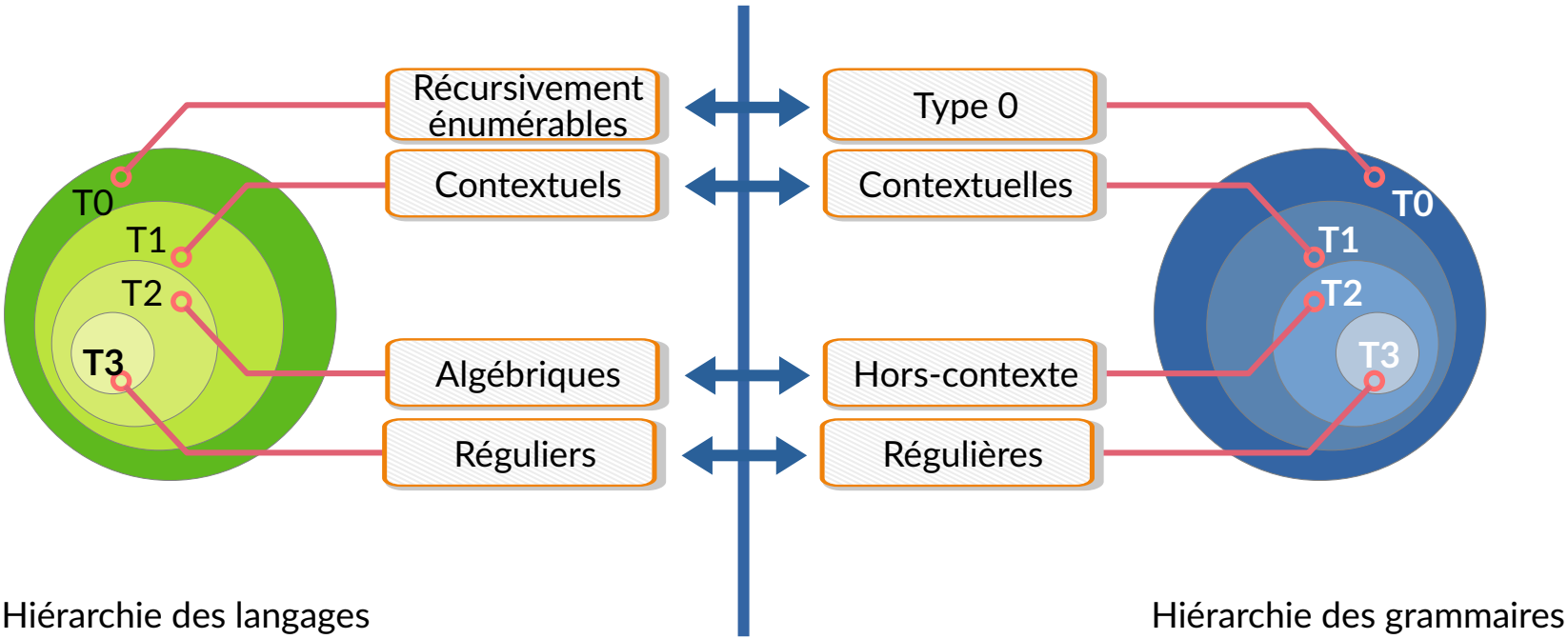
Support de cours du module théorie des langages

Expressions régulières
Equivalence des formes de représentation des langages réguliers

Présenté par Dr. T. Benouhiba

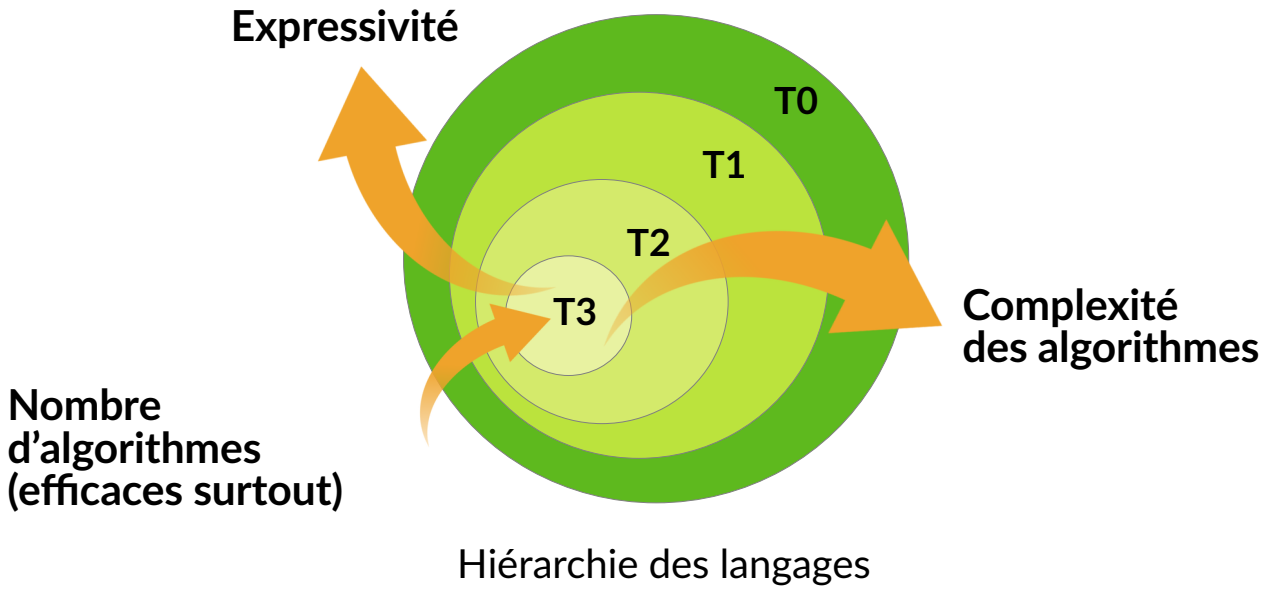


Rappel sur la hiérarchie de Chomsky





Expressivité vs. complexité





Notation par expressions régulières

- La description des caractéristiques des langages peut se faire par :
 - 1) Langage naturel : tous les mots sur $\{a,b,c\}$ ayant un nombre pair de b
 - Difficulté : le langage naturel est *ambigu*, difficile de faire un traitement automatique
 - Tous les mots $\{a,b,c\}$ qui ont peu de b !



Notation par expressions régulières

- La description des caractéristiques des langages peut se faire par :

2) Utilisation de formes à variables : $\{a^p b^q c^r \mid p, q, r \geq 0\}$

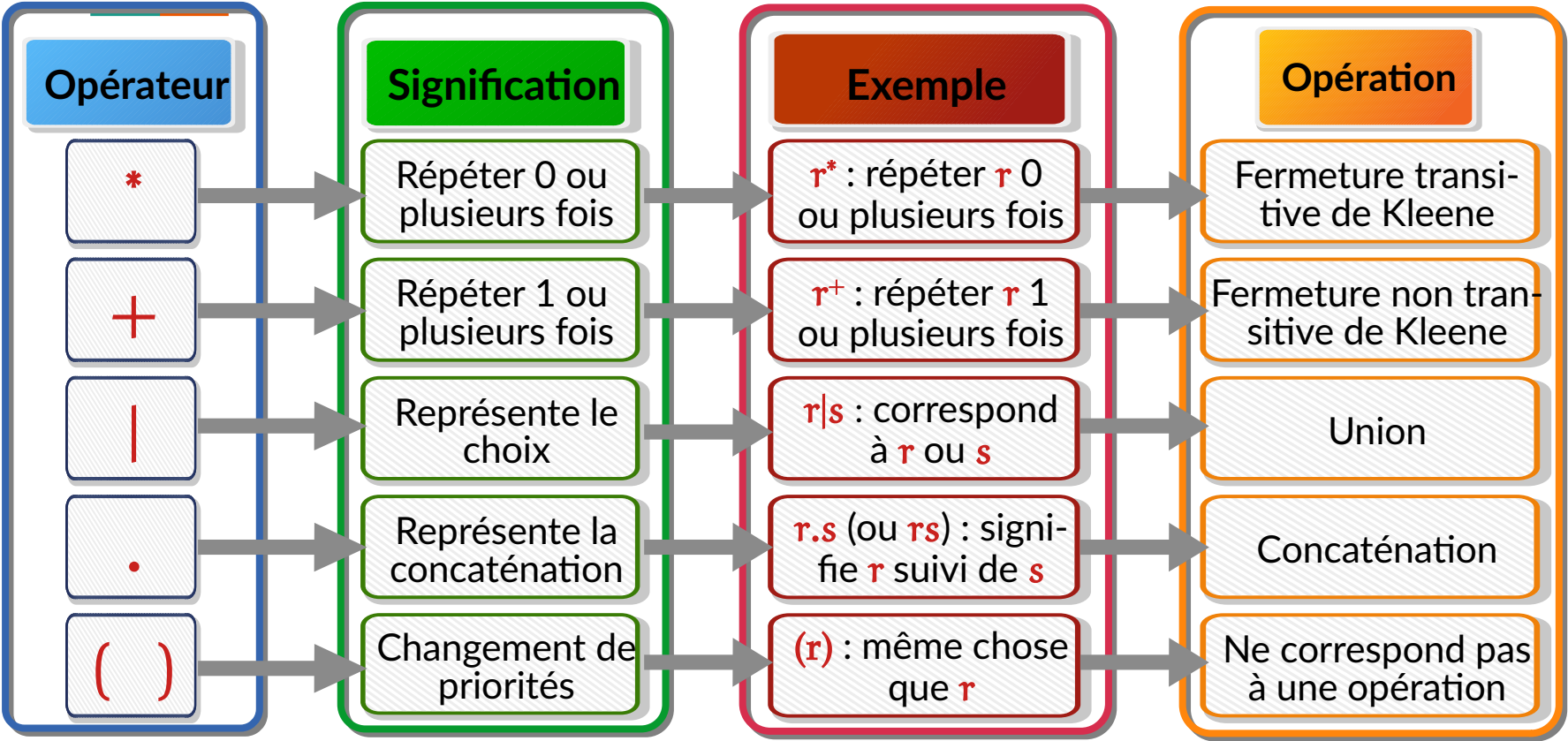
- Avantage : elle est plus précise
- Inconvénients :
 - Ne permet pas de faire la différence entre les langages selon leur complexité (en terme de la hiérarchie de Chomsky)
 - Comment peut-on faire si on a besoin d'une infinité de variables ?
 - Exemple : $\{a^p b^q c^r \mid p, q, r \geq 0\}^*$
 - Certaines formes ne s'expriment pas par des puissances (comme la forme des palindromes)



Définition d'une expression régulière

- Une expression régulière (ou *expression rationnelle*) (abréviation ER) :
 - C'est un mot formé sur l'alphabet $\{\alpha, +, *, (,), |, .\}$ (α signifie un symbole ordinaire autre que les autres symboles de cet alphabet ou ϵ), construit selon la grammaire suivante $(\{\alpha, +, *, (,), |, .\}, \{S\}, S, R)$ avec R contenant :
 - $S \rightarrow S^*$ qui signifie que si r est une ER alors r^* est une ER
 - $S \rightarrow S^+$ qui signifie que si r est une ER alors r^+ est une ER
 - $S \rightarrow S \mid S$ qui signifie que si r et s sont deux ER alors $r|s$ est une ER
 - $S \rightarrow S.S$ qui signifie que si r et s sont deux ER alors $r.s$ est une ER
 - $S \rightarrow SS$ même chose (le point est optionnel)
 - $S \rightarrow (S)$ qui signifie que si r est une ER alors (r) est une ER
 - $S \rightarrow \alpha$ qui signifie que tout mot ordinaire est une ER

Signification d'une ER





Exemples d'E.R.

- a^* : signifie répéter a 0 ou plusieurs fois, c'est le langage $\{a^n | n \geq 0\}$
- $a|b$: signifie a ou b , c'est le langage $\{a,b\}$
- $(a|b)^*$: signifie répéter a ou b 0 ou plusieurs fois, c'est donc la fermeture transitive de Kleene de $\{a,b\}$
- $(a|b)^*ab(a|b)^*$: tous les mots sur $\{a,b\}$ ayant le facteur ab
 - Peut être représentée autrement par : $b^*a^+b(a|b)^*$ ou $(a|b)^*ab^+a^*$
 - Un langage peut alors être exprimés par plusieurs ER équivalentes !



Utilisation pratique des ER

- En pratique, on utilise les ER pour :
 - Décrire des langages de programmation très simples
 - Décrire des formes simples de données
 - Exemple : format d'un champ d'un formulaire
 - Description du niveau lexical d'un langage de programmation évolué (conception des compilateurs)
 - Recherche de chaînes de caractères et substitutions des chaînes de caractères



Les ER comme des caractères jokers

- Sous Windows (explorateur ou en ligne de commande), l'ancien DOS (!!), dans certaines commandes Linux, en SQL (la construction `like`) on peut utiliser des caractères jokers pour rechercher des informations :

***** $\Rightarrow$ 0 ou plusieurs caractères quelconques

? $\Rightarrow$ un caractère quelconque

- Exemples : la commande `ls` en Linux (ou avec `dir` en ligne de commande Windows):

```
ls *.*
```

```
ls *.docx
```

```
ls *abc*.*
```

```
ls a*b?
```



Les ER POSIX

- L'utilisation des caractères jokers ne permet pas de rechercher des formes complexes
 - Rechercher un nom de fichier sans numéros
- Les ER POSIX offrent un langage beaucoup plus puissant permettant de :
 - Noter des langages réguliers avec des formes complexes
 - Noter même des langages non réguliers
- Elles sont largement utilisées dans :
 - Les commandes Linux
 - Les langages de programmation
 - Des éditeurs textes



Syntaxe de base

- `[abc]`
 - Les caractères a, b ou c
- `[a-e]`
 - Les caractères a jusqu'à e
- `[a-zA-Z]`
 - Toutes les lettres de l'alphabet standard
- `[^abc]`
 - Tous les caractères (ASCII ou UNICODE) sauf a, b et c
- `.`
 - N'importe quel caractère sans le saut de ligne (`\n`)
- `a|bc`
 - a ou le mot bc



Syntaxe de base

- a^*
 - a se répétant 0 ou plusieurs fois
- a^+
 - a se répétant 1 ou plusieurs fois
- $a?$
 - a figurant 0 ou 1 fois (donc optionnel)
- $a\{2,5\}$
 - a se répétant 2 à 5 fois (on peut utiliser $a\{2\}$ pour exactement deux fois)
- $a\{, 5\}$
 - a se répétant au plus 5 fois
- $a\{2, \}$
 - a se répétant au moins 2 fois
- $\backslash x$
 - la valeur exacte de x (x est un caractère spécial comme $\backslash, *, +, [, (,), .$, etc



Exemples

$[^a b]^*$

Tous les mots ne contenant ni a ni b

$[ab]^*$

Tous les mots sur $\{a,b\}$

$([a]^* a [a]^* a [a]^*)^*$

Tous les mots contenant un nombre pair de a

$(ab\{,4\})^*$

Tous les mots sur $\{a,b\}$ où un a ne peut être suivi de plus de 4 b



Application au contrôle des données

[0-9]{4-6}-[13][0-9]{2}-[0-9]{2}

[a-zA-Z0-9]+@[a-zA-Z0-9]+

12:00

INSCRIPTION



Nom et prénom



Mot de passe



Confirmation du mot de passe



1234-123-23



me@somewhere

Je suis :

☒ Un particulier

☐ Une entreprise

☐ J'accepte les [termes du service](#)

Valider



Utilisation en programmation

- En Java (possibilité d'utiliser la classe Pattern):

```
email.matches("[a-zA-Z0-9]+@[a-zA-Z0-9]+");
```

- En Javascript :

```
var patt = new RegExp("[a-zA-Z0-9]+@[a-zA-Z0-9]+")  
var res = patt.test(email)
```

- En Python :

```
import re  
p = re.match('[a-zA-Z0-9]+@[a-zA-Z0-9]+', email)
```




Recherche et manipulation des données

- Recherche et remplacement de texte dans un éditeur de texte (ou avec certaines commandes) :
 - Recherche et remplacement par texte brut (comme avec **notepad** de Windows)
 - Recherche : donner une chaîne à rechercher dans le texte
 - Remplacement : donner deux chaînes, l'une à chercher et l'autre à mettre à la place de la première si trouvée
 - Fonctionnalités très limitée



Recherche et manipulation des données

- Les éditeurs de texte sérieux offrent la possibilité de recherche et de remplacement basée sur les ER (atom, notepad++, **kate**, ...)
 - Recherche : fournir une ER, puis l'éditeur recherche toutes les chaînes qui correspondent à l'ER (recherche de match)
 - Remplacement : fournir deux expressions régulières
 - La première définit le format des chaînes à rechercher
 - La deuxième définit la chaîne à utiliser pour le remplacement (ce n'est pas une ER mais ce n'est pas une chaîne ordinaire non plus)
- Des outils plus performants existe pour la recherche et la manipulation des données :
 - grep, sed et surtout awk



Recherche et manipulation des données

- Utilisation des () capturer les groupes
 - Syntaxe des groupes :
 - Exemple : la chaîne « aa bbb ccc » correspond à l'ER (a+) (b+) c+
 - L'ER définit deux groupes (a+) et (b+)
 - Quand un match est établi, deux variables sont définies :
 - \1 correspond au premier groupe, il contient ici « aa »
 - \2 correspond au deuxième groupe, il contient ici « bbb »
 - Le nombre de variables disponibles dépend du nombre de groupes
 - Il y aussi la variable \0 correspondant à toute la chaîne sujette au matching
 - Utilisation des commandes \l (lowercase) et \u (uppercase) pour faire des modifications



Théorème d'équivalence des formes

- On revient aux ER mathématiques pour la théorie
- Quelle relation y a-t-il entre les ER et les langages réguliers (et les autres types aussi) ?
 - Entre les ER et les grammaires ?
 - Entre les ER et les automates ?
- Théorème de Kleene :
 - Soit Λ_{RAT} : l'ensemble des langages représentables par une ER quelconque
 - Soit Λ_{AEF} : l'ensemble des langages analysables par un AEF quelconque
 - Soit Λ_{REG} : l'ensemble des langages générés par une grammaire régulière quelconque

$$\Lambda_{\text{RAT}} = \Lambda_{\text{AEF}} = \Lambda_{\text{REG}}$$



Conséquences

- Un langage est régulier si l'une des trois conditions est vérifiée :
 - Le langage est généré par une grammaire régulière (à droite ou à gauche)
 - Le langage est analysable par un AEF
 - Le langage est représenté par une ER
- L'existence d'une forme implique l'existence des deux autres formes
- On étudiera :
 - Le passage d'une ER vers l'AEF et inversement
 - Le passage d'un AEF vers une grammaire régulière et inversement